PLONK 제약 조건 적용 방식 분석

개요

PLONK에서 제약 조건(constraints)은 회로의 정확성을 보장하는 핵심 메커니즘입니다. 이 문서는 PLONK 코드베이스에서 제약 조건이 어떻게 정의되고 적용되며 검증되는지를 분석합니다.

제약 조건의 정의

1. 게이트 제약 조건 (Gate Constraints)

PLONK에서 제약 조건은 주로 게이트(gate) 형태로 정의됩니다. 각 게이트는 특정한 수학적 관계를 강제합니다.

산술 게이트 (Arithmetic Gate)

가장 기본적인 제약 조건으로, 다음 형태의 방정식을 만족해야 합니다:

```
q_m * a * b + q_l * a + q_r * b + q_o * c + q_4 * d + q_c + PI = 0
```

여기서:

- `a`, `b`, `c`, `d` : 와이어 값 (witness values)
- `q_m`, `q_l`, `q_r`, `q_o`, `q_4`, `q_c` : 선택자 다항식 (selector polynomials)
- `PI` : 공개 입력 (public input)

범위 게이트 (Range Gate)

변수가 특정 범위 내에 있음을 보장하는 제약 조건입니다.

논리 게이트 (Logic Gate)

비트 연산(AND, XOR 등)을 구현하는 제약 조건입니다.

2. 제약 조건 트레이트 (GateConstraint Trait)

모든 게이트 타입은 `GateConstraint` 트레이트를 구현해야 합니다:

```
pub trait GateConstraint<F>
where
F: PrimeField,
{
type CustomVals: CustomValues<F>;

fn constraints(
separation_challenge: F,
wit_vals: WitnessValues<F>,
custom_vals: Self::CustomVals,
) -> F;

fn quotient_term(
selector: F,
separation_challenge: F,
wit_vals: WitnessValues<F>,
custom_vals: Self::CustomVals,
) -> F;
```

```
fn linearisation_term(
selector_polynomial: &DensePolynomial<F>,
separation_challenge: F,
wit_vals: WitnessValues<F>,
custom_vals: Self::CustomVals,
) -> DensePolynomial<F>;
}
```

제약 조건의 적용

1. 회로 구성 단계에서의 적용

StandardComposer를 통한 제약 조건 추가

```
pub fn arithmetic_gate<Fn>(&mut self, func: Fn) -> Variable
where
Fn: FnOnce(&mut ArithmeticGate<F>) -> &mut ArithmeticGate<F>,
{
// 게이트 구성
let gate = {
let mut gate = ArithmeticGate::<F>::new();
func(&mut gate).build()
};
```

```
// 선택자 벡터에 값 추가
self.q_m.push(gate.mul_selector);
self.q_l.push(gate.add_selectors.0);
self.q_r.push(gate.add_selectors.1);
self.q_o.push(gate.out_selector);
self.q_c.push(gate.const_selector);
// 와이어 벡터에 변수 추가
self.w_l.push(gate_witness.0);
self.w_r.push(gate_witness.1);
self.w_o.push(c);
self.w_4.push(w4);
// 순열 정보 업데이트
self.perm.add_variables_to_map(
gate_witness.0,
gate_witness.1,
c,
w4,
self.n,
);
self.n += 1;
c
```

```
}
```

제약 조건 타입별 적용 방식

1. **등식 제약 조건 (Equality Constraints)**

```
pub fn assert_equal(&mut self, a: Variable, b: Variable) {
self.poly_gate(
a,
b,
self.zero_var,
F::zero(), // q_m = 0
F::one(), // q_l = 1
-F::one(), // q_r = -1
F::zero(), // q_o = 0
F::zero(), // q_c = 0
None,
);
}
```

2. **상수 제약 조건 (Constant Constraints)**

```
pub fn constrain_to_constant(
&mut self,
a: Variable,
```

```
constant: F,
pi: Option<F>,
) {
self.poly_gate(
a,
a,
a,
F::zero(), // q_m = 0
F::one(), // q_l = 1
F::zero(), // q_r = 0
F::zero(), // q_o = 0
-constant, // q_c = -constant
pi,
);
}
```

2. 증명 생성 단계에서의 적용

몫 다항식(Quotient Polynomial) 계산

제약 조건들은 몫 다항식 계산에서 결합됩니다:

```
fn  compute_gate_constraint_satisfiability<F, P>(
domain: &GeneralEvaluationDomain<F>,
range_challenge: F,
```

```
logic_challenge: F,

fixed_base_challenge: F,

var_base_challenge: F,

prover_key: &ProverKey<F>,

wl_eval_8n: &[F],

wr_eval_8n: &[F],

wo_eval_8n: &[F],

w4_eval_8n: &[F],

pi_poly: &DensePolynomial<F>,

) -> Result<Vec<F>, Error> {

Ok((0..domain_8n.size())

.map(|i| {

let  wit_vals = WitnessValues {

a_val: wl_eval_8n[i],

b_val: wr_eval_8n[i],

c_val: wo_eval_8n[i],

d_val: w4_eval_8n[i],

};



// 각 게이트 타입별 제약 조건 계산

let  arithmetic = prover_key.arithmetic.compute_quotient_i(i, wit_vals);

let  range = Range::quotient_term(/* ... */);

let  logic = Logic::quotient_term(/* ... */);
```

```
let fixed_base_scalar_mul = FixedBaseScalarMul::quotient_term(/* ... */);

let curve_addition = CurveAddition::quotient_term(/* ... */);



// 모든 제약 조건 결합

(arithmetic + pi_eval_8n[i])

+ range

+ logic

+ fixed_base_scalar_mul

+ curve_addition

})

.collect())

}
```

제약 조건의 검증

1. 회로 만족도 검사 (Circuit Satisfaction Check)

개발 단계에서 회로가 올바르게 구성되었는지 확인하기 위한 검사:

```
pub fn check_circuit_satisfied(&mut self) {

for i in 0..self.n {

// 선택자 값들 가져오기

let qm = self.q_m[i];

let ql = self.q_l[i];
```

```
let qr = self.q_r[i];

let qo = self.q_o[i];

let qc = self.q_c[i];

let q4 = self.q_4[i];

let qarith = self.q_arith[i];

let qrange = self.q_range[i];

let qlogic = self.q_logic[i];

// 와이어 값들 가져오기

let a = w_l[i];

let b = w_r[i];

let c = w_o[i];

let d = w_4[i];

let pi = pi_vec[i];



// 산술 게이트 제약 조건 검사

let arithmetic_constraint = qarith * (

(qm * a * b) +

(ql * a) +

(qr * b) +

(qo * c) +

(q4 * d) +

pi +

q_hl * a.pow([SBOX_ALPHA]) +
```

```
q_hr * b.pow([SBOX_ALPHA]) +

q_h4 * d.pow([SBOX_ALPHA]) +

qc

);


// 범위 게이트 제약 조건 검사

let  range_constraint = qrange * (

delta(*c - four * d) +

delta(*b - four * c) +

delta(*a - four * b) +

delta(*d_next - four * a)

);


// 논리 게이트 제약 조건 검사

let  logic_constraint = qlogic * (/* 논리 게이트 제약 조건 */);


// 전체 제약 조건이 0이어야 함

let  total_constraint = arithmetic_constraint + range_constraint + logic_constra

assert_eq!(total_constraint, F::zero(), "Check failed at gate {}", i);

}

}
```

2. 증명 검증 단계에서의 검증

검증자는 다음과 같은 방식으로 제약 조건을 검증합니다:

1. **선형화 다항식 (Linearisation Polynomial) 계산**

- 각 게이트 타입별로 선형화 기여도 계산
- 모든 제약 조건을 하나의 다항식으로 결합

2. **몫 다항식 검증**

- 제약 조건들이 vanishing polynomial로 나누어떨어지는지 확인
- 다항식 약정을 통한 검증

순열 제약 조건 (Permutation Constraints)

순열 제약 조건의 역할

순열 제약 조건은 서로 다른 게이트의 와이어들이 같은 값을 가져야 함을 보장합니다.

순열 맵핑

```
pub fn add_variables_to_map(
&mut self,
a: Variable,
b: Variable,
c: Variable,
d: Variable,
gate_index: usize,
) {
let left: WireData = WireData::Left(gate_index);
```

```
let right: WireData = WireData::Right(gate_index);

let output: WireData = WireData::Output(gate_index);

let fourth: WireData = WireData::Fourth(gate_index);



// 각 변수를 해당 와이어에 매핑

self.add_variable_to_map(a, left);

self.add_variable_to_map(b, right);

self.add_variable_to_map(c, output);

self.add_variable_to_map(d, fourth);

}
```

순열 다항식 계산

순열 제약 조건은 별도의 누적기 다항식(accumulator polynomial) Z(X)를 통해 검증됩니다.

룩업 제약 조건 (Lookup Constraints)

룩업 제약 조건은 특정 값들이 미리 정의된 테이블에 포함되어 있음을 보장합니다.

룩업 테이블 구성

```
pub struct LookupTable<F> {

table_1: MultiSet<F>,

table_2: MultiSet<F>,

table_3: MultiSet<F>,
```

```
table_4: MultiSet<F>,

}
```

룩업 제약 조건 검증

룩업 제약 조건은 Plookup 프로토콜을 사용하여 검증됩니다.

제약 조건 최적화

1. 선택자 최적화

- 사용되지 않는 게이트의 선택자는 0으로 설정
- 여러 제약 조건을 하나의 게이트로 결합

2. 커스텀 게이트

- 특정 연산에 최적화된 커스텀 게이트 구현
- 제약 조건 수 감소를 통한 성능 향상

결론

PLONK의 제약 조건 시스템은 다음과 같은 특징을 가집니다:

1. **모듈화된 설계**: 각 게이트 타입이 독립적으로 구현되고 결합됨
2. **유연한 제약 조건**: 다양한 타입의 제약 조건을 지원
3. **효율적인 검증**: 선형화와 몫 다항식을 통한 효율적인 검증
4. **확장 가능성**: 새로운 게이트 타입을 쉽게 추가할 수 있는 구조

이러한 설계를 통해 PLONK는 복잡한 회로에 대해서도 효율적인 영지식 증명을 생성할 수 있습니다.